

RAG Foundations

Chunking · Embeddings · Vector Stores · Hybrid Retrieval · Re-ranking · RAGAS

WEEK 3 · DAY 15

Monday, July 20 2026

Day 15 of 84

Sec	Topic	Coverage
Sec 1	RAG Architecture	Why RAG, 3-phase pipeline (index/retrieve/generate)
Sec 2	Chunking Strategies	Fixed-size, recursive, semantic, document-aware
Sec 3	Embedding Models	BGE, E5, OpenAI, Cohere; model selection guide
Sec 4	Vector Stores	Qdrant, pgvector, FAISS, Pinecone, Milvus, Weaviate
Sec 5	Retrieval Strategies	Dense, BM25, hybrid + Reciprocal Rank Fusion
Sec 6	Re-ranking	Cross-encoder, Cohere Rerank; two-stage pipeline
Sec 7	RAGAS Evaluation	faithfulness, answer_relevancy, context_recall, precision
Sec 8	Advanced RAG	HyDE, lost-in-the-middle, query decomposition
Sec 9	Capstone Piece 2	SchemaRAG class: index tables, retrieve top-k
Sec 10	FAANG Patterns	Microsoft/Amazon/Google/Meta/Netflix production RAG
Sec 11	Architect's Lens	Failure taxonomy, chunking guide, model selection
Sec 12	Exercises + Summary	Build schema RAG, measure recall@k, compare strategies

CAPSTONE PIECE 2 OF 8 — Schema-Aware RAG: Retrieve only relevant table definitions per question. Enables Text-to-SQL on schemas with 50+ tables without cramming all definitions into every prompt. Reduces prompt tokens by 10× and improves model focus on relevant tables.


```
    chunk_size=1000, chunk_overlap=200,
    separators=["\n\n", "\n", ". ", " ", ""], # tries each in order
)
# Splits on paragraph breaks first (most semantic), falls back to sentence,
# word, character. Respects natural text boundaries. Best general-purpose choice.
```

Strategy 3: Semantic Chunking

```
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer
import numpy as np

def semantic_chunk(sentences: list[str], threshold: float=0.35) -> list[str]:
    model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
    embs = model.encode(sentences)
    chunks, current = [], [sentences[0]]
    for i in range(1, len(sentences)):
        sim = cosine_similarity(embs[i-1:i], embs[i:i+1])[0][0]
        if sim < threshold: # topic changed → split here
            chunks.append(" ".join(current))
            current = [sentences[i]]
        else:
            current.append(sentences[i])
    if current: chunks.append(" ".join(current))
    return chunks

# Best for: heterogeneous documents, research papers. Requires embedding every sentence.
```

Strategy 4: Document-Aware (SQL Schema)

```
import re

def chunk_schema(schema_sql: str) -> list[dict]:
    """Each table = one chunk. Preserves the natural semantic unit for SQL retrieval."""
    tables = re.findall(r'(CREATE TABLE\s+(\w+)\s*\([^;]+\s*);',
                        schema_sql, re.DOTALL | re.IGNORECASE)
    return [{"table": name, "sql": sql.strip()} for sql, name in tables]

# For Text-to-SQL: 50-table schema → 50 chunks (one per table)
# Query "users who bought product X" retrieves users + order_items tables
# Model sees 2 tables, not all 50. 10× token reduction.
```

Strategy	Best For	Chunk Size	Overlap	Quality
Fixed-size	Uniform docs, code	512 tokens	64 tok	Fast, predictable
Recursive	General text (best default)	1000 chars	200 ch	Good
Semantic	Mixed-topic docs, research	Variable	None	Highest, slow
Document-aware	SQL schemas, APIs, FAQs	Per entity	None	Best for structured

Section 3 — Embedding Models for Retrieval

The embedding model maps text to vectors. The retrieval quality ceiling is determined by the embedding model's ability to capture semantic similarity. BGE models add an instruction prefix for query embedding that improves asymmetric retrieval (query is short; document is longer).

```
from sentence_transformers import SentenceTransformer
import numpy as np

class EmbeddingService:
    def __init__(self, model_name: str = "BAAI/bge-large-en-v1.5"):
        self.model = SentenceTransformer(model_name)
        self.dim = self.model.get_sentence_embedding_dimension() # 1024

    def embed_documents(self, texts: list[str], batch_size: int=64) -> np.ndarray:
        return self.model.encode(texts, batch_size=batch_size,
                                   normalize_embeddings=True,
                                   show_progress_bar=len(texts) > 1000)

    def embed_query(self, query: str) -> np.ndarray:
        # BGE instruction prefix: improves query-to-passage matching
        query = f"Represent this sentence for searching relevant passages: {query}"
        return self.model.encode(query, normalize_embeddings=True)

# normalize_embeddings=True → L2 normalised → cosine sim = dot product
# (faster and numerically equivalent to cosine when normalised)
```

Model	Dim	Speed	Quality	Cost
all-MiniLM-L6-v2	384	Very fast	Good	Free (self-hosted)
BAAI/bge-large-en-v1.5	1024	Medium	Excellent	Free (self-hosted)
intfloat/e5-mistral-7b-instruct	4096	Slow	Best OSS	Free (needs GPU)
openai/text-embedding-3-small	1536	Fast	Excellent	\$0.02/1M tokens
openai/text-embedding-3-large	3072	Fast	Best API	\$0.13/1M tokens
cohere/embed-english-v3.0	1024	Fast	Excellent	\$0.10/1M tokens
amazon/titan-embed-text-v2	1024	Fast	Excellent	\$0.02/1M tokens

Section 4 — Vector Stores in Production

Qdrant — recommended for production self-hosted:

```
from qdrant_client import QdrantClient
from qdrant_client.models import (Distance, VectorParams, PointStruct,
                                   Filter, FieldCondition, MatchValue)

import uuid

client = QdrantClient(url="http://localhost:6333")
```

```

client.create_collection("sql_schemas",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE))

# Index chunks
emb = EmbeddingService()
points = [
    PointStruct(id=str(uuid.uuid4()),
        vector=emb.embed_documents([c["sql"]])[0].tolist(),
        payload={"table": c["table"], "sql": c["sql"],
            "schema_name": "ecommerce"})
    for c in schema_chunks
]
client.upsert(collection_name="sql_schemas", points=points)

# Retrieve with metadata filter
def retrieve_tables(question: str, schema: str, top_k: int=5) -> list[dict]:
    q_vec = emb.embed_query(question).tolist()
    hits = client.search("sql_schemas", query_vector=q_vec,
        query_filter=Filter(must=[
            FieldCondition(key="schema_name", match=MatchValue(value=schema))]),
        limit=top_k, with_payload=True)
    return [{"table": h.payload["table"], "sql": h.payload["sql"],
        "score": h.score} for h in hits]

```

pgvector — best if already running PostgreSQL:

```

# One-time setup:
# CREATE EXTENSION IF NOT EXISTS vector;
# CREATE INDEX ON schema_chunks USING ivfflat (embedding vector_cosine_ops)
# WITH (lists = 100);

def pgvector_retrieve(question: str, schema_name: str, top_k: int=5, conn=None):
    q_vec = EmbeddingService().embed_query(question).tolist()
    cur = conn.execute("""
        SELECT table_name, sql,
            1 - (embedding <=> %s::vector) AS score
        FROM   schema_chunks
        WHERE  schema_name = %s
        ORDER BY embedding <=> %s::vector
        LIMIT  %s
        """, (q_vec, schema_name, q_vec, top_k))
    return [{"table": r[0], "sql": r[1], "score": r[2]} for r in cur.fetchall()]
# <=> is the cosine distance operator in pgvector

```

Store	Deploy	Best For	ANN Index	Filtering
FAISS	In-process	Research / small scale	IVF, HNSW	No (manual pre-filter)
Qdrant	Docker / Cloud	Production serving	HNSW	Yes (payload filters)
Weaviate	Docker / Cloud	Multi-modal + hybrid	HNSW	Yes (GraphQL)

Pinecone	Managed SaaS	Zero-ops teams	Proprietary	Yes (metadata)
pgvector	PostgreSQL ext	Already on Postgres	IVFFlat / HNSW	Yes (SQL WHERE)
Milvus	Docker / K8s	Billion-scale	IVF, HNSW, DiskANN	Yes

Section 5 — Retrieval Strategies: Dense, Sparse, Hybrid

Dense retrieval handles semantic similarity. Sparse (BM25) handles exact keyword matching. Hybrid combines both — always beats pure dense in enterprise settings.

```
# BM25 sparse retrieval
from rank_bm25 import BM25Okapi

class BM25Retriever:
    def __init__(self, docs: list[str]):
        self.bm25 = BM25Okapi([d.lower().split() for d in docs])
        self.docs = docs
    def retrieve(self, query: str, top_k: int=10) -> list[tuple[str,float]]:
        scores = self.bm25.get_scores(query.lower().split())
        top_i = scores.argsort()[::-1][:top_k]
        return [(self.docs[i], float(scores[i])) for i in top_i]

# Strength: exact keywords, product codes, table/column names
# Weakness: no semantic understanding (synonym mismatch)

# Reciprocal Rank Fusion – hybrid combination
def reciprocal_rank_fusion(dense: list[tuple[str,float]],
                           sparse: list[tuple[str,float]], k: int=60) -> list[tuple[str,float]]:
    """
    RRF score = 1/(k + rank) summed across retrieval methods.
    k=60 (Cormack et al. 2009 default).
    Higher k = less penalty for lower rank.
    """
    scores: dict[str,float] = {}
    for rank, (doc, _) in enumerate(dense):
        scores[doc] = scores.get(doc, 0) + 1.0 / (k + rank + 1)
    for rank, (doc, _) in enumerate(sparse):
        scores[doc] = scores.get(doc, 0) + 1.0 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)

# Qdrant native hybrid (production):
# client.search() with sparse_vector + fusion="rrf" parameter
```

Section 6 — Re-ranking with Cross-Encoders

A bi-encoder (embedding model) compares query and chunk independently — fast but shallow. A cross-encoder sees both simultaneously — accurate but 10-100× slower. Two-stage pipeline: fast bi-encoder retrieves top-20, cross-encoder re-ranks to top-5.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query: str, candidates: list[str], top_n: int=5) -> list[str]:
    pairs = [[query, doc] for doc in candidates]
    scores = reranker.predict(pairs) # one forward pass per pair
    ranked = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
    return [doc for doc, _ in ranked[:top_n]]

# Two-stage production pipeline:
```

```
# Step 1: Dense ANN → top 20 (fast: ~5ms, Qdrant HNSW)
# Step 2: Cross-encoder rerank → top 5 (slower: ~50ms, GPU)
# Step 3: Prompt with top 5 → LLM

# Cohere Rerank managed API:
import cohere
co = cohere.Client("YOUR_KEY")
def cohere_rerank(query: str, documents: list[str], top_n: int=5):
    resp = co.rerank(query=query, documents=documents,
                     model="rerank-english-v3.0", top_n=top_n)
    return [{"index":r.index,"score":r.relevance_score,
            "document":documents[r.index]} for r in resp.results]
```


Section 7 — RAG Evaluation with RAGAS

RAGAS measures four orthogonal dimensions of RAG quality. All four use LLM-as-judge internally. Run on a curated test set of 50-100 question/context/answer triplets.

```
from ragas import evaluate
from ragas.metrics import (faithfulness, answer_relevancy,
                           context_recall, context_precision)
from datasets import Dataset

eval_data = {
    "question": ["Which users never placed an order?"],
    "answer": ["SELECT u.email FROM users u LEFT JOIN orders o "
              "ON u.user_id=o.user_id WHERE o.order_id IS NULL"],
    "contexts": [
        ["CREATE TABLE users (user_id INT, email TEXT ...)"],
        ["CREATE TABLE orders (order_id INT, user_id INT ...)"]],
    "ground_truth": ["SELECT u.email FROM users u LEFT JOIN orders o "
                     "ON u.user_id=o.user_id WHERE o.order_id IS NULL"],
}

result = evaluate(Dataset.from_dict(eval_data),
                  metrics=[faithfulness, answer_relevancy,
                           context_recall, context_precision])

print(result)

# Production targets for SQL RAG:
# faithfulness      > 0.90 (no hallucinated column names)
# answer_relevancy  > 0.85
# context_recall    > 0.85 (right tables must be retrieved)
# context_precision > 0.75 (some noise acceptable)
```

Metric	Measures	Score 1.0 Means	LLM Judge?
Faithfulness	Answer stays within retrieved context	Zero hallucination	Yes
Answer Relevancy	Answer addresses the question	Perfectly on-topic	Yes
Context Recall	Context covers the gold-standard answer	Full coverage of answer info	Yes
Context Precision	Fraction of retrieved chunks that were useful	Zero noise chunks	Yes

Section 8 — Advanced RAG Techniques

HyDE — Hypothetical Document Embeddings

Generate a hypothetical answer first, embed it, retrieve using that vector. A hypothetical SQL query is closer in embedding space to real table definitions than the natural language question is. Gains +3-8% context_recall.

```
async def hyde_retrieve(question: str, llm_client, retriever, top_k: int=10):
    # Step 1: generate hypothetical SQL (use a fast/cheap model)
```

```

resp = await llm_client.post("/v1/chat/completions", json={
    "model": "gpt-4o-mini",
    "messages": [{ "role": "system", "content": "Generate a plausible SQL query." },
                  { "role": "user", "content": question } ],
    "max_tokens": 200, "temperature": 0.5,
})
hypo_sql = resp.json()["choices"][0]["message"]["content"]
# Step 2: embed the SQL answer (not the question)
q_vec = emb_service.embed_query(hypo_sql)
# Step 3: retrieve using the hypothetical embedding
return retriever.search(q_vec, top_k=top_k)

```

Lost-in-the-Middle Mitigation

```

# LLMs best attend to START and END of context.
# Middle positions are underweighted (Liu et al. 2023).
def arrange_for_attention(chunks: list[str]) -> list[str]:
    """chunks[0] = most relevant. Reorder: most relevant at start + end."""
    if len(chunks) <= 2: return chunks
    return [chunks[0]] + chunks[2:] + [chunks[1]]
# chunks[0] = most relevant → position 0 (start of context)
# chunks[1] = 2nd most relevant → final position (end of context)
# chunks[2:] = less relevant → middle (where attention fades)

```

Section 9 — Capstone Piece 2: SchemaRAG Class

CAPSTONE PIECE 2: Schema-Aware RAG — retrieve only the relevant table definitions for each query. BEFORE: all 50 tables in every prompt (15,000 tokens). AFTER: 5 tables retrieved per query (1,500 tokens). 10x token reduction.

```
class SchemaRAG:
    def __init__(self, qdrant_url: str="http://localhost:6333"):
        self.client = QdrantClient(url=qdrant_url)
        self.embedder = SentenceTransformer("BAAI/bge-large-en-v1.5")
        self._ensure_collection()

    def index_schema(self, schema_name: str, schema_sql: str):
        """Parse schema SQL, embed each table definition, store in Qdrant."""
        tables = re.findall(r'(CREATE TABLE\s+(\w+)\s*\([^\)]+\))',
                           schema_sql, re.DOTALL | re.IGNORECASE)

        points = []
        for full_sql, table_name in tables:
            enriched = f"Table: {table_name}\n{full_sql.strip()}"
            vec = self.embedder.encode(enriched, normalize_embeddings=True).tolist()
            points.append(PointStruct(
                id=str(uuid.uuid4()), vector=vec,
                payload={"schema_name": schema_name, "table_name": table_name,
                        "sql": full_sql.strip()}))
        self.client.upsert(COLLECTION, points=points)
        print(f"Indexed {len(points)} tables for schema '{schema_name}'")

    def retrieve_tables(self, question: str, schema_name: str,
                       top_k: int=5) -> list[dict]:
        q_text = f"Represent this sentence for searching relevant passages: {question}"
        q_vec = self.embedder.encode(q_text, normalize_embeddings=True).tolist()
        hits = self.client.search(COLLECTION, query_vector=q_vec,
                                  query_filter=Filter(must=[FieldCondition(
                                      key="schema_name", match=MatchValue(value=schema_name))]),
                                  limit=top_k, with_payload=True)
        return [{"table": h.payload["table_name"], "sql": h.payload["sql"],
                  "score": h.score} for h in hits]

    def build_schema_context(self, question: str, schema_name: str,
                             top_k: int=5) -> str:
        """Returns only the relevant CREATE TABLE statements for this question."""
        tables = self.retrieve_tables(question, schema_name, top_k)
        return "\n".join(t["sql"] for t in tables)

# In the Day 14 FastAPI endpoint:
schema_rag = SchemaRAG()
schema_rag.index_schema("ecommerce", FULL_SCHEMA_SQL) # index once at startup

async def generate_sql_with_rag(question: str, schema_name: str) -> dict:
    schema_context = schema_rag.build_schema_context(question, schema_name, top_k=5)
    return await call_vllm(question, schema_context) # only relevant tables in prompt
```

Section 10 — FAANG Production RAG Patterns

Microsoft — Azure AI Search Hybrid RAG (Bing Copilot)

Copilot uses Azure AI Search with BM25 + Ada-002 embeddings fused via RRF. Enterprise RAG (GitHub Copilot for Business, 365 Copilot): each tenant's docs in an isolated namespace within a shared Qdrant/Cognitive Search cluster. Re-ranking: cross-encoder fine-tuned on enterprise search click data. Key insight: hybrid ALWAYS beats pure dense for enterprise — product names, SKUs, and employee IDs are exact-match queries that dense retrieval misses.

Amazon — Kendra + Bedrock RAG (Amazon Q)

Amazon Q uses Amazon Kendra (purpose-built enterprise search) for retrieval: understands document structure (headers, tables, FAQs), 40+ data connectors (Salesforce, Confluence, SharePoint), incremental indexing. Retrieval feeds into Amazon Bedrock (Claude-based generation). Amazon's research: document-aware chunking (FAQ entries as atomic units) outperforms fixed-size chunking by 15-20% on enterprise Q&A; benchmarks.

Google — Vertex AI Search + Gemini RAG

Vertex AI Search uses a dual-encoder trained on web-scale + enterprise data. Combines embeddings with Knowledge Graph signals: 'GOOG' disambiguates to Alphabet Inc. and boosts the entity cluster in retrieval. For code workloads (internal developer tools): function-level chunking — each function definition is one chunk, preserving imports and docstrings. Dramatically improves retrieval precision for code search vs. fixed-size.

Meta — RA-DIT: Joint Retriever + Generator Fine-tuning

RA-DIT (Retrieval-Augmented Dual Instruction Tuning) jointly fine-tunes BOTH the embedding model and the generation model on (question, gold passage) pairs. The retriever learns to find passages the generator finds HELPFUL (not just similar). This bidirectional loop — retriever improves generator, generator feedback improves retriever — is how Meta's production RAG for Llama outperforms RAG with off-the-shelf embeddings. Same principle for Text-to-SQL: fine-tune the schema embedder on (question, relevant_table) pairs for best recall.

Netflix — Content Metadata RAG for Recommendation

Netflix RAG powers conversational content recommendations. Corpus: 15,000+ titles, each with logline, genres, cast, themes (LLM-extracted), user engagement signals. Chunking: title-level (one chunk per title). Retrieval: hybrid — dense on theme/mood embeddings + BM25 on title/cast names (users search by actor name). Re-ranking: cross-encoder incorporating personalisation signals (user watch history). Generation synthesises 3-5 retrieved titles into a conversational recommendation.

Section 11 — Architect's Lens: Failure Taxonomy

RAG FAILURE MODE TAXONOMY — WHERE TO LOOK WHEN ANSWERS ARE WRONG:

'Answer is completely wrong / hallucinated'

→ Faithfulness < 0.7

→ Is retrieved context relevant? YES → generation prompt issue
NO → retrieval issue (see below)

'Answer is missing key information'

→ Context recall < 0.7

→ Is the right chunk in the index? NO → indexing pipeline gap
YES → embedding/chunking mismatch

'Answer uses context but gets it wrong'

→ Faithfulness high but correctness low

→ Generation prompt is too vague → add few-shot examples

'Performance is slow'

→ ANN search: should be < 20ms for 1M vectors (Qdrant HNSW)

→ Reranking: cross-encoder adds 40-100ms for 20 candidates

→ Embedding: GPU = <1ms/doc; CPU = ~10ms/doc → use batching

→ Total RAG overhead vs. direct LLM: +50-150ms → acceptable

CHUNKING SELECTION GUIDE:

Uniform docs (PDFs, articles) → Recursive, 1000 chars, 200 overlap

FAQs / structured Q&A → Document-aware (one per Q&A pair)

Code files → Function-level (one per function)

SQL schemas → Table-level (one per CREATE TABLE)

Long research papers → Semantic chunking

Real-time data / live databases → Query at retrieval time (SQL RAG)

EMBEDDING MODEL GUIDE:

Self-hosted, best quality → BAAI/bge-large-en-v1.5 (1024d, free)

Multilingual → intfloat/multilingual-e5-large (1024d)

Managed API, best quality → openai/text-embedding-3-large (3072d)

Low latency on CPU → all-MiniLM-L6-v2 (384d, very fast)

Domain-specific (SQL/code) → Fine-tune bge-large on domain pairs

Section 12 — Hands-On Exercises + Summary

Exercise 1: Build the Schema RAG Index

```
# 1. Start Qdrant
docker run -p 6333:6333 qdrant/qdrant

# 2. Index schema and test retrieval
rag = SchemaRAG()
rag.index_schema("ecommerce", FULL_SCHEMA_SQL)

for q in ["How many users from USA?",
          "Average order value?",
          "Products never purchased?",
          "Revenue per user per country"]:
    tables = rag.retrieve_tables(q, "ecommerce", top_k=3)
    print(f"Q: {q}")
    for t in tables:
        print(f"    {t['table']} ({t['score']:.3f})")
```

Exercise 2: Measure Recall@k

```
gold = {
    "How many users from USA?": {"users"},
    "Average order value?": {"orders"},
    "Products never purchased?": {"products", "order_items"},
    "Revenue per user per country": {"users", "orders"},
}
for q, gold_tables in gold.items():
    retrieved = {t["table"] for t in rag.retrieve_tables(q, "ecommerce", top_k=3)}
    recall = len(gold_tables & retrieved) / len(gold_tables)
    print(f"Recall@3: {recall:.2f} | {q[:50]}")
# Target: Recall@3 >= 0.85 for all questions
```

Summary

Concept	Key Point
RAG motivation	Knowledge cutoff + hallucination + context economics
Indexing pipeline	Chunk → Embed → Store (offline)
Chunking	Most impactful decision; table-level for SQL schemas
Embedding models	BGE-large (free, high quality) or text-embedding-3-large
Vector stores	Qdrant (production self-hosted), pgvector (on Postgres)
Dense retrieval	Semantic similarity; handles paraphrases
Sparse (BM25)	Exact keywords; handles product codes, column names
Hybrid + RRF	Always beats pure dense in enterprise settings
Re-ranking	Cross-encoder: +5-15% precision, 40-100ms cost

RAGAS	faithfulness, answer_relevancy, context_recall, context_precision
HyDE	Embed hypothetical answer → +3-8% context_recall
Lost-in-the-middle	Place most relevant chunks at start AND end of context
Capstone Piece 2	SchemaRAG: index tables, retrieve top-k, 10× token reduction

NEXT: Day 16 — Production RAG Indexing Pipelines

Incremental indexing, document update strategies, metadata filtering, multi-tenant schema registry, scaling the index to handle live schema changes, and embedding model fine-tuning for SQL domain.